Milestone 6, Part 4a: Measuring the Blueprint

holdem-cfr walkthrough

Why this needs its own machinery

On Kuhn, Leduc and the postflop subgames, quality had an exact answer: exploitability, computed by a best response. For the full no-limit blueprint there is no exact answer on a laptop. Computing a best response would mean walking the entire real game against the blueprint's strategy, and that is out of reach. So we fall back on the other standard of evidence in computer poker: **play matches, and report results with error bars**.

The difficulty with matches is noise. A single hand of no-limit can swing a hundred big blinds, and skill differences are a few hundredths of a big blind per hand. Naively, telling a strong bot from a mediocre one takes millions of hands. Two classic tools cut that down.

Duplicate play. Every deal is played twice, with the seats swapped. If one agent is dealt aces against kings, the other agent gets the aces in the mirror hand. Card luck shows up on both sides and largely cancels, so what remains is mostly the difference in *decisions*. Bridge tournaments have used this for a century. In code, `duplicate_match` samples a deal, plays it with A in seat 0 and B in seat 1, plays it again with B in seat 0 and A in seat 1, and records A's total over the pair.

Confidence intervals over pairs. The pairs are independent, so the mean has standard error σ/√n, and the 95% interval is ±1.96 standard errors. Results are in **milli-big-blinds per hand** (mbb/hand), the literature's unit: +100 mbb/hand means winning a tenth of a big blind every hand.

Checking the harness before trusting it

A measuring instrument needs its own calibration, so the first test does two things.

- **Random against random.** Two random agents must break even within the interval: +124 ± 435 mbb/hand over 40,000 hands, comfortably consistent with zero.
- **A deterministic agent against itself.** This must score *exactly* zero. In duplicate play each copy holds both seats of every deal, so every chip one copy wins in the first hand it loses in the mirror. The test asserts equality, not closeness, which pins down the seat-swapping and sign bookkeeping completely.

The baselines

All agents choose from the same abstract action menu, so the blueprint never sees a bet size outside its tree. Opponents that bet arbitrary amounts need action translation, which is the next part.

- **Always call:** never folds, never raises. It is the classic "calling station", the first thing any value-betting strategy should crush. - **Always raise:** takes the smallest raise whenever it can. This tests whether the blueprint folds weak hands to pressure and re-raises strong ones. - **Random:** uniform over the menu. - **Equity heuristic:** a sensible rule-based player. It estimates equity against a random hand by Monte Carlo, raises above 72% equity, bets above 60% when checked to, calls when the price is right with a 5-point margin, and folds otherwise. It is the most serious baseline here, because it plays the obvious things correctly.

Results

A deliberately weak blueprint shows the harness has resolving power. It uses the made-hand-category abstraction (it cannot tell a flush draw from air) and only 1.5 million iterations:

Opponent	Result (mbb/hand)	Hands
always call	+1,613 ± 258	40,000
always raise	+2,785 ± 374	40,000
random	+2,125 ± 268	40,000
equity heuristic	−1,289 ± 462	8,000

It crushes the three naive bots and **loses** clearly to the equity heuristic. That is exactly what such a crude abstraction should do, and it shows the harness can tell agents apart in both directions. The CI test requires the three wins with intervals excluding zero.

The real abstraction (169/50/50/50 E[HS²] buckets) trained for six minutes on the 2-core VM, 34 million iterations, gives a different picture:

Opponent	Result (mbb/hand)	Hands
always call	+1,428 ± 75	100,000
always raise	+4,011 ± 199	100,000
random	+2,169 ± 152	100,000
equity heuristic	**+619 ± 180**	20,000
the same trainer after 6 million iterations	**+106 ± 30**	400,000

Two results matter here. First, the real buckets turn the −1,289 against the equity heuristic into **+619**. The E[HS²] abstraction is what lets the blueprint understand draws, kickers and relative hand strength. Second, a longer-trained blueprint beats a shorter-trained copy of itself by a clear margin. That is direct evidence that the 6-minute run was still improving, and that the 400-million-iteration laptop run should be stronger still.

These numbers are wins against fixed, exploitable opponents and against an earlier self. They are **not** a measure of how close the blueprint is to equilibrium. A strategy can beat every baseline here and still be very exploitable by an opponent who targets its leaks. The lower bound on that exploitability, local best response, is the next measurement.

Reproduce

```
cargo run --release -p cli -- blueprint --minutes 20 --linear-minutes 5
cargo run --release -p cli -- arena --pairs 100000
# Compare two checkpoints (did more training help?):
cp data/blueprint.bin data/blueprint_20min.bin
cargo run --release -p cli -- blueprint --minutes 40          # resumes and keeps training
cargo run --release -p cli -- arena --vs data/blueprint_20min.bin
```

A note on the MacBook Air run

The 20-minute run on the MacBook Air held about 500,000 iterations per second for the first six minutes, then dropped to between 200,000 and 350,000, with occasional stalls. The Air has no fan, so a sustained load on all cores pushes it into thermal throttling. The run still completed 408 million iterations. `--threads 6` trades a little peak speed for less throttling, and shorter repeated runs also work, since the trainer resumes from its checkpoint.